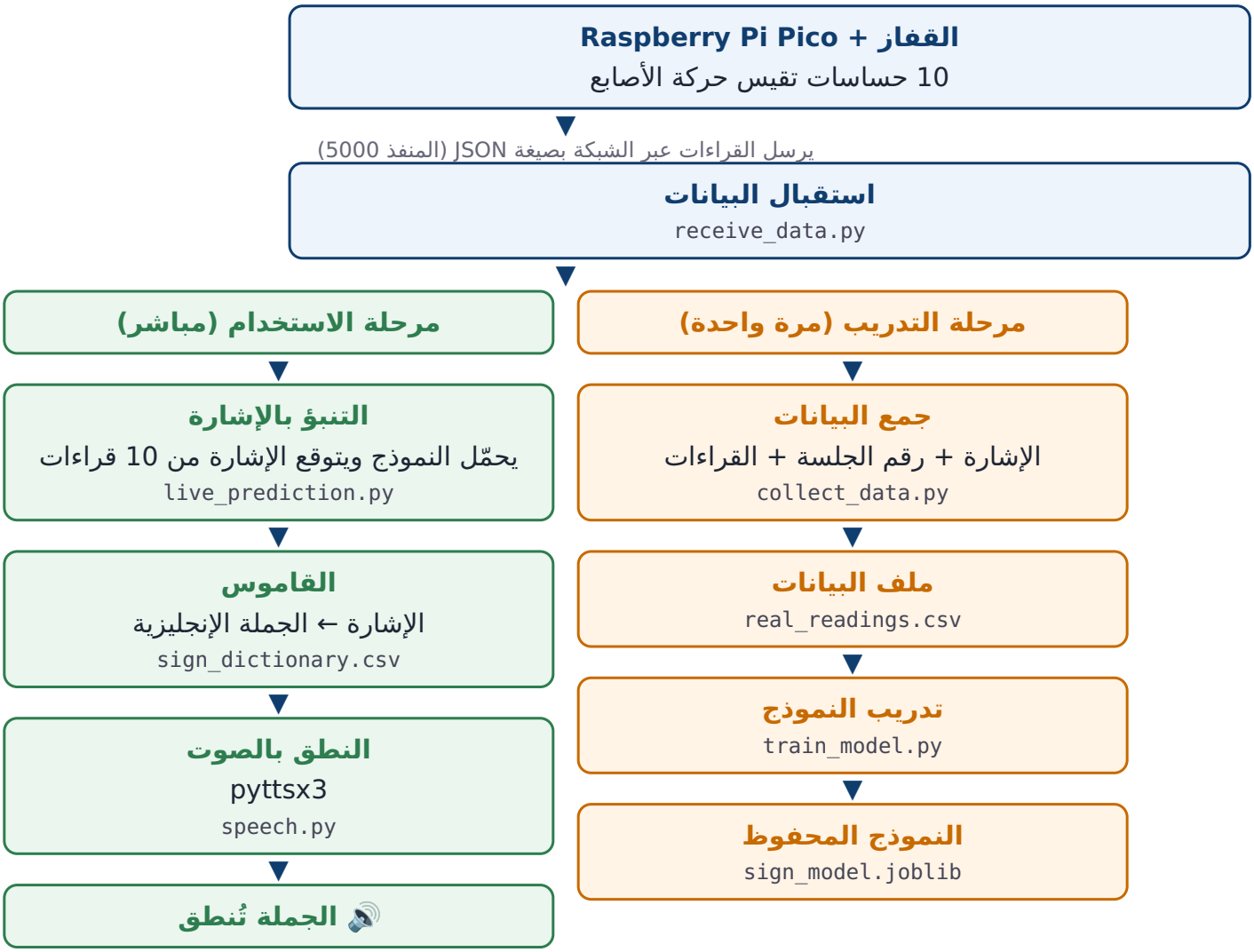
قفاز لغة الإشارة الذكي

شرح تدفق التطبيق: من حساسات القفاز إلى الصوت

١. الفكرة العامة

القفاز فيه **10 حساسات** متصلة بـ Raspberry Pi Pico. تُرسَل قراءات الحساسات إلى الحاسوب، فيتعرف نموذج ذكاء اصطناعي (Random Forest) على **الإشارة** (مثل `pain`)، ثم تتحول إلى **جملة إنجليزية منطوقة**.

٢. التدفق الكامل



٣. وظيفة كل ملف

الملف	وظيفته
`collect_data.py`	تكتب اسم الإشارة و10 قراءات يدويًا، فيحفظها صفًا في `real_readings.csv` مع `session_id`.
`train_model.py`	ينظف البيانات، ويقسمها حسب **الجلسة** (حتى لا يُختبر النموذج على جلسة تدرّب عليها)، ويدرب Random Forest، ويطبع الدقة، ويحفظ النموذج.
`receive_data.py`	سيرفر TCP ينتظر الـ Pico ويستقبل القراءات، لكنه حاليًا **يطبعها فقط**.
`live_prediction.py`	يحمّل النموذج ويتوقع الإشارة من 10 قراءات (تُدخل يدويًا حاليًا).
`speech.py`	يبحث عن الجملة الإنجليزية للإشارة في `sign_dictionary.csv` وينطقها.

٤. شكل البيانات

كل صف في `real_readings.csv` هو قراءة واحدة للحساسات العشرة:

العمود	المعنى
`timestamp`	وقت التسجيل
`sensor_1 … sensor_10`	قراءات الحساسات (مدخلات النموذج)
`label`	اسم الإشارة الحقيقية (ما نريد أن يتعلمه النموذج)
`session_id`	رقم جلسة التسجيل (لتقسيم التدريب والاختبار)

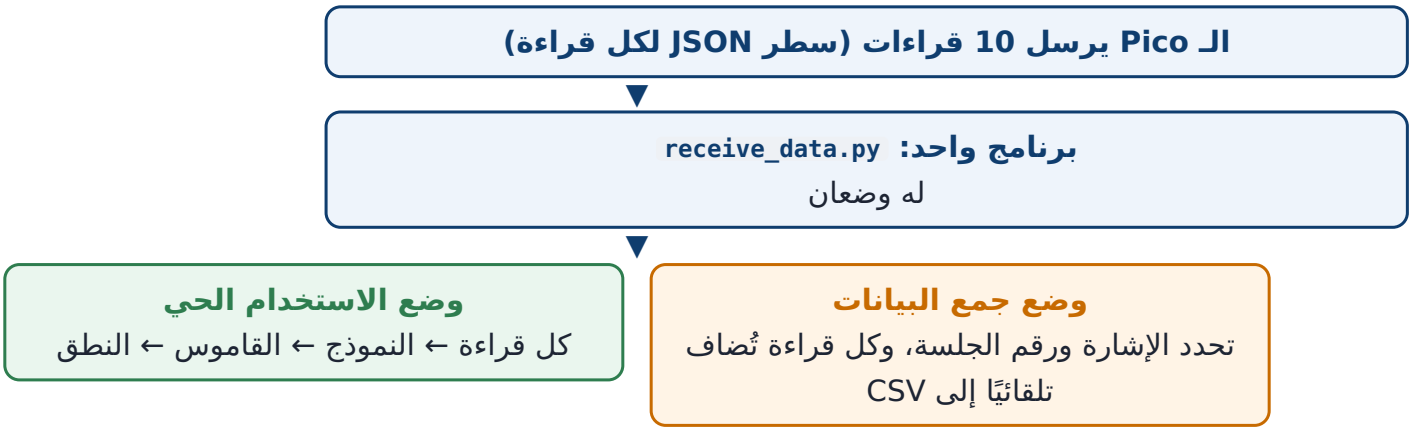
> **قاعدة التدريب:** كل إشارة يجب أن تُسجَّل في **عدة جلسات مختلفة** (4 إلى 5 على الأقل)، وفي كل جلسة عشرات القراءات.

٥. المشاكل الحالية في المشروع

1. **الأجزاء غير مربوطة:** `receive_data.py` لا يحفظ القراءات ولا يمررها للنموذج، و`live_prediction.py` و`collect_data.py` يعتمدان على الإدخال اليدوي.
2. **جمع البيانات يدوي:** إدخال 10 أرقام لكل صف، لذلك الملفات فيها صف أو صفان فقط.
3. **اختلاف ترتيب الأعمدة:** `collect_data.py` يكتب `session_id` بعد `timestamp`، بينما الملفات القديمة تضعه في النهاية. إضافة صفوف جديدة إلى ملف قديم تُزيح القيم بين الأعمدة.
4. **مسار التدريب:** `train_model.py` يقرأ من `test_data` لا `data`.
5. **الجلسات مكررة:** في `real_readings_baec82.csv` إشارتان مختلفتان في الجلسة نفسها `1`.
6. **القيم ثابتة (100):** لا يستطيع النموذج التعلم منها.

7. **استقبال TCP:** `recv(1024)` قد يستقبل رسالة مقطوعة أو رسالتين ملتصقتين. الحل أن تنتهي كل رسالة بسطر جديد.
8. **القاموس:** `speech.py` يقرأ بفاصل Tab، ويجب أن تطابق الإشارات فيه الـ labels المستخدمة في التدريب.

٦. التدفق المقترح بعد الإصلاح



٧. خطوات العمل بالترتيب

1. تعديل `receive_data.py` ليحفظ القراءات تلقائيًا (وضع الجمع).
2. تسجيل 4 إلى 5 جلسات لكل إشارة، بقراءات متغيرة فعلًا.
3. التأكد من أن `train_model.py` يقرأ من مجلد `data` ثم تدريب النموذج.
4. التأكد من أن `sign_dictionary.csv` فيه كل الإشارات بنفس أسمائها.
5. ربط `receive_data.py` بـ `predict_sign` ثم `speak` (وضع الاستخدام الحي).